



## IA en Código Heredado, README, Buenas Prácticas y Pipeline

Ejercicio de Clase · Ejercicio 3 de 3 · Bloques 3.6 — 3.7 — 3.8 — Pipeline Integrado

|                                               |                 |
|-----------------------------------------------|-----------------|
| <b>Nombre del alumno/a</b>                    | Angel Echenique |
| <b>Fecha</b>                                  | 03/04/2026      |
| <input type="checkbox"/> <b>Enlace GitHub</b> |                 |

☐ Instrucciones: Responde con lo que vimos en clase. Puedes consultar tus apuntes. En el Bloque 5 deberás investigar por tu cuenta y citar la fuente. ¡Solo demostrar que entendiste, no hay que escribir código!

### BLOQUE 1 — IA explicando código heredado (3.7)

Vimos que la IA puede analizar y documentar código heredado. Clasifica los tipos de código heredado que puede analizar y los resultados que puede producir:

| La IA puede ANALIZAR código... | La IA puede GENERAR...               |
|--------------------------------|--------------------------------------|
| 1. Código sin comentarios      | 1. Explicaciones en lenguaje natural |
| 2. Variables mal nombradas     | 2. Nombres más descriptivos          |
| 3. Funciones complejas         | 3. Resúmenes paso a paso             |
| 4. Código desordenado          | 4. Docstrings estructurados          |
| 5. Lógica desconocida          | 5. Diagramas de flujo o pseudocódigo |



## Generación de Documentación IA Código Heredado



### 1.2 — Analiza este código heredado con la IA

Observa este código heredado sin documentar. Responde las preguntas de análisis como si fueras la IA:

```
def f(a, b):  
    c = []  
    for i in range(len(a)):  
        if i < len(b):  
            c.append(a[i] * b[i])  
        else:  
            c.append(a[i])  
    return sum(c) / len(c)
```

| Pregunta de analisis              | Respuesta(como si fueras IA)                                                                                        |
|-----------------------------------|---------------------------------------------------------------------------------------------------------------------|
| ¿Qué hace esta función?           | Multiplica elementos de dos listas posición por posición (si existen en ambas) y calcula el promedio del resultado. |
| ¿Qué son los parámetros a y b?    | Listas numéricas (arrays) que contienen valores a procesar.                                                         |
| ¿Qué devuelve exactamente?        | Un número decimal (float) que representa la media de los valores calculados.                                        |
| ¿Qué problemas potenciales tiene? | No valida tipos, puede fallar si las listas están vacías (división por 0), y usa nombres poco descriptivos.         |
| ¿Cómo renombrarías f, a, b, c?    | f → calcular_promedio_combinado, a → lista1, b → lista2, c → resultados                                             |



### 1.3 — El diagrama conceptual en texto

La IA puede crear diagramas conceptuales en texto sin herramientas externas. Escribe el diagrama de flujo en texto para la función analizada:

*Ejemplo de formato: [Entrada: lista a y lista b] → [Combinar según longitud] → [Calcular media] → [Resultado float]*

[Entrada: lista1 y lista2]  
→ [Recorrer lista1]  
→ [Multiplicar elementos si existe en lista2]  
→ [Si no, mantener valor original]  
→ [Guardar resultados]  
→ [Calcular promedio]  
→ [Resultado float]



## Generación de Documentación IA Código Heredado



### BLOQUE 2 — Generación de README con IA (3.6)

Un buen README profesional generado por IA tiene secciones específicas. Para cada sección, describe su contenido y escribe un ejemplo breve:

| Sección      | ¿Qué incluye?        | Ejemplo                                                   |
|--------------|----------------------|-----------------------------------------------------------|
| Descripción  | Qué hace el proyecto | "CaféBot es un chatbot para tomar pedidos en cafeterías." |
| Instalación  | Cómo instalar        | <code>pip install -r requirements.txt</code>              |
| Uso          | Cómo ejecutar        | <code>python app.py</code>                                |
| Ejemplos     | Casos de uso         | "Usuario: quiero un café →Bot responde..."                |
| Estructura   | Carpetas y módulos   | <code>/src, /tests, /docs</code>                          |
| Dependencias | Librerías usadas     | Flask, requests                                           |
| Licencia     | Tipo de licencia     | MIT License                                               |



## 2.2 — Escribe el prompt para generar el README

Tienes un proyecto Python llamado 'CaféBot' — un chatbot para tomar pedidos de cafetería. Escribe el prompt completo para pedir a la IA un README profesional:

Actúa como un experto en documentación técnica.

Genera un README profesional para un proyecto en Python llamado 'CaféBot', que es un chatbot para tomar pedidos en cafeterías.

Quiero que contenga

- Descripción del proyecto
- Instalación
- Uso
- Ejemplos de interacción
- Estructura del repositorio
- Dependencias
- Licencia

- Formato: Markdown listo para GitHub

Pd: Proporciona el contenido del proyecto o estructura del repositorio como contexto para mejorar la calidad de la documentación.



## Generación de Documentación IA Código Heredado



### BLOQUE 3 — Buenas prácticas y errores comunes (3.8)

Vimos el checklist de buenas prácticas al documentar con IA. Completa la tabla:

| Buenas Prácticas ✓              | Errores Comunes ✗              |
|---------------------------------|--------------------------------|
| Revisar documentación generada  | Pedir documentación sin código |
| Especificar contexto en prompts | Usar prompts vagos             |
| Definir audiencia               | No adaptar el nivel            |
| Usar formatos estructurados     | No definir formato             |
| Validar coherencia con código   | Aceptar errores de la IA       |

### 3.2 — El flujo IA + Humano + Pipeline

Vimos en clase que el flujo de trabajo profesional tiene 3 fases. Completa la tabla:

| Fase          | Responsable | Acción principal      | Tareas clave (al menos 2)                                                                      |
|---------------|-------------|-----------------------|------------------------------------------------------------------------------------------------|
| 1. Generación | IA          | Crear la base         | <ul style="list-style-type: none"><li>➤ Generar docstrings</li><li>➤ Explicar código</li></ul> |
| 2. Validación | Humano      | Criterio y seguridad  | <ul style="list-style-type: none"><li>➤ Corregir errores</li><li>➤ Validar seguridad</li></ul> |
| 3. Ejecución  | Pipeline    | Garantía y despliegue | <ul style="list-style-type: none"><li>➤ Tests automáticos</li><li>➤ Deploy</li></ul>           |



## Generación de Documentación IA Código Heredado



### BLOQUE 4 — Integración completa: Documentación + Pipeline + Modularización

Conectamos los 3 módulos aprendidos: modularización, pipeline CI/CD y documentación con IA. Responde cómo se relacionan:

| Módulo aprendido                   | ¿Qué aporta a la documentación con IA? | Ejemplo concreto de integración         |
|------------------------------------|----------------------------------------|-----------------------------------------|
| Modularización de código           | Código más claro y documentable.       | Separar en módulos facilita docstrings. |
| Pipeline CI/CD (GitHub Actions)    | Automatiza documentación.              | GitHub Actions genera docs.             |
| Documentación con IA (este módulo) | Genera contenido.                      | Docstrings, README, OpenAPI.            |

#### 4.2 — Orden correcto del flujo de trabajo

Ordena del 1 al 8 el flujo de trabajo completo cuando un desarrollador hace un cambio en el código:

| Orden | Acción en el flujo de trabajo                                        |
|-------|----------------------------------------------------------------------|
| 1     | El desarrollador modifica una función en un módulo                   |
| 2     | El desarrollador hace git push y se abre un Pull Request             |
| 3     | La IA analiza el código y detecta vulnerabilidades o malas prácticas |
| 4     | IA genera o actualiza el docstring de las funciones modificadas      |



## Generación de Documentación IA Código Heredado



| Orden | Acción en el flujo de trabajo                                     |
|-------|-------------------------------------------------------------------|
| 5     | El desarrollador revisa y valida la documentación generada por IA |
| 6     | El pipeline ejecuta los tests automáticos                         |
| 7     | La IA genera el reporte técnico con la documentación actualizada  |
| 8     | El pipeline hace el deploy si todo está correcto                  |

### 4.3 — Caso práctico final

□ Proyecto real: Trabajas en un equipo que mantiene un sistema de facturación con 5 años de antigüedad. El código tiene 200 funciones, cero comentarios, nombres de variables como 'x', 'tmp', 'data2', y ningún README. El cliente pide que el sistema esté completamente documentado en 2 semanas. Tienes acceso a IA y a un pipeline CI/CD con GitHub Actions.

Diseña tu estrategia completa. ¿Por dónde empezarías? ¿Qué herramientas y tipos de documentación priorizarías? ¿Cómo automatizarías el proceso en el pipeline?

#### Estrategia:

1. Día 1-3
  1. Usar IA para analizar código heredado
  2. Renombrar variables
  3. Generar docstrings
2. Día 4-7



## Generación de Documentación IA Código Heredado



1. Generar documentación API
2. Crear README automático
3. Día 8-14

1. Integrar en CI/CD
2. Automatizar documentación

### Herramientas:

- IA (Copilot, ChatGPT, Claude, OpenCode)
- OpenAPI
- GitHub Actions

### Automatización:

- Script que genera docstrings en commits.
- Pipeline que actualiza README.
- Generación automática de documentación web.

## BLOQUE 5 — Investigación propia (cita la fuente)

□ Investiga: Este bloque requiere que investigues por tu cuenta. Usa una fuente fiable y cita siempre de dónde lo sacaste.

### 5.1 — Herramienta para documentar código heredado con IA



## Generación de Documentación IA Código Heredado



Investiga una herramienta o extensión que use IA para entender o documentar código legacy/heredado (ej: GitHub Copilot, Sourcegraph Cody, Amazon CodeWhisperer, Tabnine...):

| Campo                 | Respuesta                                                                             |
|-----------------------|---------------------------------------------------------------------------------------|
| Herramienta           | GitHub Copilot                                                                        |
| ¿Cómo analiza código? | Interpreta contexto del código y sugiere explicaciones.                               |
| ¿Qué genera?          | Comentarios, docstrings, explicaciones.                                               |
| ¿IDE?                 | Sí (VS Code)                                                                          |
| CI/CD                 | Sí, integrable.                                                                       |
| Fuente                | <a href="https://github.com/features/copilot">https://github.com/features/copilot</a> |
| Fecha                 | 03/04/2026                                                                            |

### Ejemplo práctico: documentación con IA (GitHub Copilot)

Caso: código heredado sin documentación

Código legacy (sin comentarios)

```
<> Python
def calc(a, b, op):
    if op == "add":
        return a + b
    elif op == "sub":
        return a - b
    elif op == "mul":
        return a * b
    elif op == "div":
        return a / b if b != 0 else None
```

Uso de GitHub Copilot (IA)



## Generación de Documentación IA Código Heredado



### Copilot analiza:

- nombres de variables (a, b, op)
- estructura condicional
- operaciones matemáticas
- contexto del archivo

```
</> Python

def calc(a, b, op):
    """
    Realiza operaciones básicas entre dos números.

    Args:
        a (float): primer número
        b (float): segundo número
        op (str): tipo de operación:
            - 'add' suma
            - 'sub' resta
            - 'mul' multiplicación
            - 'div' división

    Returns:
        float | None: resultado de la operación o None si división por cero
    """
```

### Impacto real en este ejemplo

- Código sin documentación → entendimiento lento
- Con IA → explicación inmediata
- Reduce tiempo de análisis de legacy code

## 5.2 — Impacto real de la IA en la documentación técnica



## Generación de Documentación IA Código Heredado



Busca un estudio, informe o artículo técnico que mida el impacto de usar IA para generar documentación (tiempo ahorrado, calidad, adopción en empresas...):

| Campo      | Respuesta                                             |
|------------|-------------------------------------------------------|
| Artículo   | GitHub Copilot Productivity Study.                    |
| Métricas   | Hasta 55% más rápido programando.                     |
| Conclusión | La IA mejora productividad.                           |
| Fuente     | <a href="https://github.blog">https://github.blog</a> |
| Fecha      | 03/04/2026                                            |

### Ejemplo práctico: impacto de IA en documentación técnica

**Caso:** equipo en empresa trabajando con código legacy

**Situación sin IA**

Un desarrollador tarda:

- 30 min → entender función antigua
- 20 min → escribir documentación manual
- 10 min → revisar consistencia

Total: **60 minutos**

**Con IA (GitHub Copilot / similares)**

- 10 min → revisión del código
- 10 min → ajuste de docstring generado por IA

Total: **20 minutos**

**Comparación directa (basado en el estudio de GitHub Copilot)**



## Generación de Documentación IA Código Heredado



| Métrica                 | Sin IA | Con IA          |
|-------------------------|--------|-----------------|
| Tiempo de documentación | 60 min | 20 min          |
| Velocidad               | Base   | +55% más rápido |
| Esfuerzo cognitivo      | Alto   | Medio-bajo      |

### Analisis final de la actividad

Esta actividad demuestra cómo la inteligencia artificial puede integrarse en todo el ciclo de vida del software, desde el análisis de código heredado hasta la automatización completa de la documentación en pipelines CI/CD.

Módulo 3 · Documentación Técnica con IA · Dicampus